



Agentic Repository-Level Code Migration for Small Language Models

Duc-Hanh Dang · Tu-Duc Nguyen

University of Engineering and Technology, VNU Hanoi

hanhdd@vnu.edu.vn · 23021534@vnu.edu.vn

2026-10-06

Contents

- 1** Introduction
- 2** Methodology
- 3** Inspirations
- 4** Benchmarks
- 5** References

Introduction

Code migration

Migration moves a working codebase to a new language. The new code must do the same job and pass the same tests.

- Many systems still run on legacy C, Java, and Go.
- Hand migration is slow. Every module repeats the same steps: read, plan, translate, test.
- The steps are regular. Regular work is where automation helps.

Why automate with LLM agents

1. **Scale.** A repository has hundreds of modules. One developer cannot iterate that fast.
2. **Context.** No single prompt holds a whole repository. Agents split the work and share state.
3. **Proof.** A model claim is not evidence. An agent can run the target toolchain and report counts.

Zhu, Y., Liu, L., Yu, J., & Zhang, D. (2026). LLM-based multi-agent orchestration: A survey of frameworks, communication protocols, and emerging patterns. *Future Internet*, 18(6), 326.

ARCMiS

Agentic **R**epository-level **C**ode **M**igration for **S**mall language models.

- A harness that migrates a whole source codebase to a target language.
- Hierarchical multi-agent orchestration over small models: 27B parameters, one local GPU.
- Every result is scored by the target toolchain, never by a model claim.
- Languages are config. This campaign targets C to Rust.

One module, three translations

C (real source)

```
int chtrie_walk(chtrie *tr, int from,
               int sym, int creat)
{
    struct chtrie_edge *p;
    unsigned long h;
    h = (unsigned long)from*tr->alphsz
        + sym;
    h %= tr->ecap;
    for (p = tr->etab[h]; p; p = p->next)
        if (p->from == from
            && p->sym == sym)
            return p->to;
    ...
}
```

c2rust-style mechanical output (illustrative)

```
pub unsafe extern "C" fn chtrie_walk(
    tr: *mut chtrie, from: c_int,
    sym: c_int, creat: c_int)
-> c_int {
    let mut p: *mut chtrie_edge;
    let mut h: c_ulong;
    h = (from as c_ulong)
        .wrapping_mul((*tr).alphsz
                     as c_ulong)
        .wrapping_add(sym as c_ulong);
    h %= (*tr).ecap;
    p = (*tr).etab.offset(h as isize);
    ...
}
```

ARCMiS output (real, verified)

```
pub fn chtrie_walk(
    tr: &mut ChTrie, from: usize,
    sym: usize, creat: bool)
-> Result<usize, ChTrieError> {
    let h = ((from as u64)
             * (tr.alphsz as u64)
             + sym as u64)
            % tr.ecap as u64;
    let mut p = tr.etab[h as usize]
                .as_deref();
    while let Some(e) = p {
        if e.from == from
            && e.sym == sym {
            return Ok(e.to);
        }
        p = e.next.as_deref();
    }
    ...
}
```

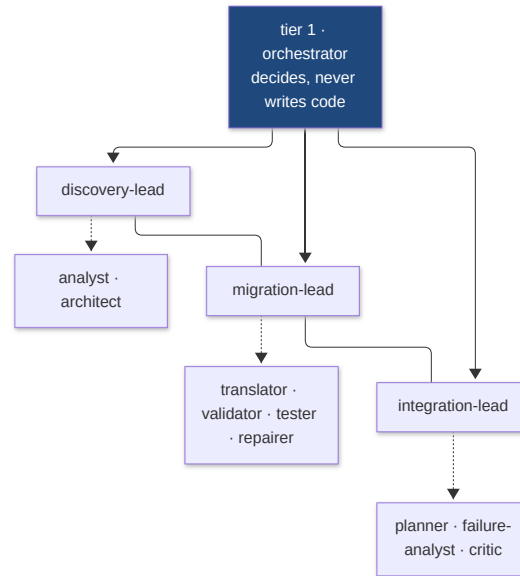
C: `tool_projects/crust/chtrie/src/chtrie.c`, function `chtrie_walk`. Rust: real ARCMiS workspace, round s33, cargo test 10 passed / 0 failed. The mechanical form keeps raw pointers and unsafe. The ARCMiS form keeps the algorithm and hands the compiler the checks.

Methodology

The methodology in four parts

- 1 Hierarchy.** A tier-1 orchestrator, tier-2 leads, tier-3 specialists.
- 2 Rounds.** A ten-phase state machine with evidence gates.
- 3 State.** A shared blackboard, a guard on every tool call, hard budgets.
- 4 Proof.** Toolchain-only scoring over a coverage sweep.

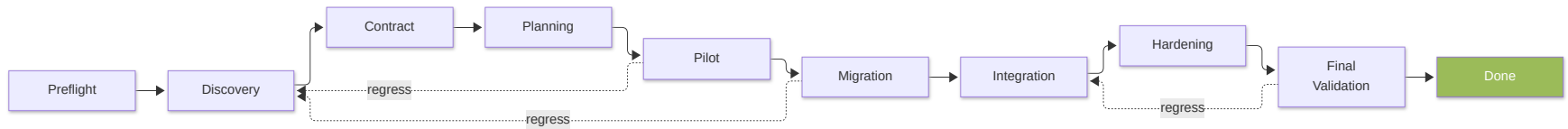
The agent hierarchy



- Tier 2 leads resolve from config (`mas.hierarchy`), not code.
- A tier-1 delegation naming a specialist is refused (`deny_direct`).

ADR 0022, ADR 0026. Code: `lib/orchestrator/src/hierarchy.rs` , `lead.rs` .

The ten-phase state machine



- A phase exit needs evidence on disk. A claim alone never passes a gate.
- Contract needs `brief.md` with a validator pass. FinalValidation needs the toolchain at exit zero.
- Regressions are legal and recorded.

Phase table: project report, \$methodology (verified against `docs/report/paper.tex`).

The blackboard

One shared workspace per run. The state stays small by design:

File	Cap	Holds
<code>plan.md</code>	4000 chars	the current plan
<code>notes.md</code>	8000 chars	cross-agent notes
<code>state.json</code>	counter-capped	phase, counters
<code>tasks.json</code>	task list	task-graph mirror
<code>decisions.jsonl</code>	append	every decision

- The model never sees the whole history. Snapcompact renders old turns to PNG frames and keeps a short recent window (12k token threshold, 4k recent).
- Character caps stop one agent from filling the shared state for all others.

Code: `lib/blackboard` , `lib/snapcompact` . Budgets: project report, `$blackboard`.

The guard

Every tool call from every tier passes one deterministic gateway:

- **Allowlist.** A role may call only its tools.
- **Path policy.** `source/` stays read-only. Reads are scoped against the context window.
- **Deny patterns.** Shell escape calls are refused with feedback, and repeats feed the breaker.
- **Ask arbitration.** A denied `ask` call may go to the jev judge. Every other denial is final. Without a judge, the `ask` slot stays deny-by-default.

ADR 0023. Code: `lib/orchestrator/src/guard.rs` , `guard_hook.rs` , `jev_judge.rs` .

Budgets and breakers

Campaign defaults (`scripts/round-template.yml`):

Budget	Value	Breaker	Trigger
max rounds	20	stagnation	3 rounds, no completed task
orchestrator turns	20	round cap	20 rounds, run closes
worker turns	40	output cap	8192 tokens per answer
judge turns	40	per-role override	16384 (translator, architect, repairer, orchestrator)
fanout	3	engine faults	backoff, then retry

- A budget stops one member. A breaker stops the run and writes the ledger row.
- In this campaign the breaker was the graceful exit, not a failure.

Source: `scripts/round-template.yml` ; stagnation rule in project report, `$budgets`.

Scoring: only the toolchain decides

`scripts/rescore.py` runs after every round:

1. Run the target build command.
2. Run the target test command.
3. Write `result/per_problem.json` with counts.

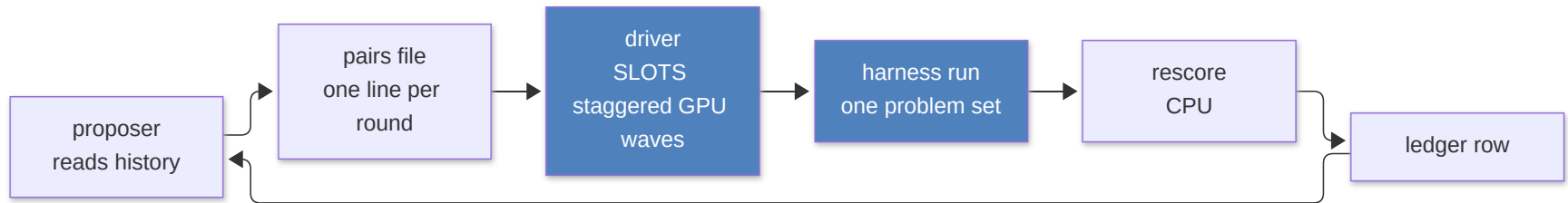
Verdict	Meaning
SOLVED	build green, tests pass
NOT CLEARED	closed without a verified workspace
ABORTED	engine failure, no verdict earned
IN FLIGHT	still running

A model claim is never a verdict.

A problem counts as cleared when one round leaves a workspace the toolchain accepts. Retries count the problem once.

Code: `scripts/rescore.py` . This campaign scores Rust targets with cargo.

The coverage sweep pipeline



- GPU lane: one set at a time, LoC ascending, SLOTS concurrent rounds staggered apart.
- CPU lane: proposer and scorer run while the GPU grinds.
- One candidate, one config delta, one problem set. The ledger keeps cause and effect.

Code: `scripts/sweep.sh` , `scripts/rounds_ledger.py` .

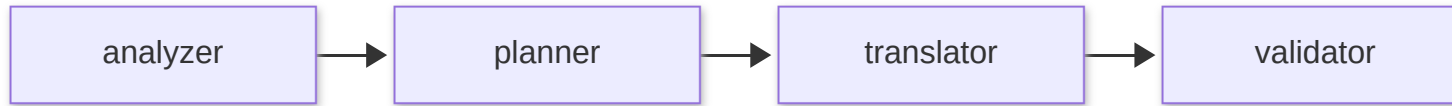
Inspirations

Orchestration patterns: two axes

	static agents	dynamic-adaptive
centralized	one boss, fixed roles	one boss, routing at runtime
hierarchy	fixed tree of teams	ARCMiS sits here

- Agents stay static: one preamble, one tool allowlist, one judge contract each.
- Selection is dynamic: a task graph re-scores after each delegation, and a failing task can spawn a failure-analyst chain.

ReCodeAgent: the pipeline discipline



- A multi-agent workflow for repository-level translation and validation, run with a frontier model over 118 projects.
- ARCMiS borrows two things: the `tool_projects` dataset (crust, oxidizer, alphasat, skel) and the stage discipline. Stages map onto our phases. The pipeline stays config, not hardcoded control flow.

Claude Modernization Plugin: gated execution



- A field workflow for modernizing a codebase in three passes.
- Each pass exits only on evidence: the brief exists, the tests ran.
- ARCMiS borrows the gate idea: a phase exit needs files, not a claim.

Why judgments cost turns

Every classification that a busy LLM makes is one decode on the busy GPU:



- The harness makes hundreds of small judgments per run: did a member fail, is a tool call in scope, does a round need help.
- Each rung down the ladder removes decode work from the main model.

Ladder per Li, J., Miao, C., Krishnan, S., & Padman, R. (2026). JEV-as-a-judge: Accept when confident, escalate when unsure. arXiv:2609.26550.

jev in ARCMiS

- `jev_judge.rs` . Arbitrates denied `ask` calls in the guard. One laya choice question, threshold 0.9. Below the threshold, the denial stands (ADR 0023).
- `jev_triage.rs` . Triages every lead member dispatch. One forward pass answers five `agent_trace_observability` questions. A min-confidence cascade accepts confident verdicts and falls back on the rest (ADR 0027).

Honest state: the round policy wires triage as an observer (`round_triage/policy.rs`). The harness records every verdict. No round stops on one. Enforcement (`policy: enforce`) exists and is not the campaign default (ADR 0028).

Typed decisions: laya

One small checkpoint (ModernBERT-large, 421M parameters) answers constrained questions in a single forward pass:

Question type	Answer
choice	one label, with probabilities
score	a rubric level
noul	a true-or-false need

- No token decoding. Microseconds to seconds per call, on CPU.
- In ARCMiS it backs the judge and the triage. The checkpoint is a specialist: off-workflow questions fall back to base behavior, so the harness checks the trained label sets before it trusts a verdict.

Benchmarks

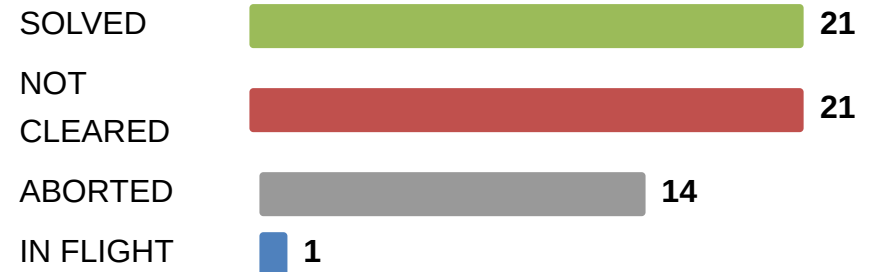
Setup

Item	Value	Budget	Value
model	qwen3.8-27B, local	max rounds	20
engine	ninfer :8081, RTX 3090	orchestrator turns	20
context	num_ctx 65536	worker turns	40
output	8192 tokens (four roles: 16384)	judge turns	40
temperature	0.2	stagnation breaker	3 rounds
fanout	3		
Family	Source language	Problems	
crust	C	100	
skel	Python	8	
oxidizer	Go	7	
alr	scripts/round-template.yml, assets/ReCodeAgent/data/tool_projects	4	

- Counted on disk: 119 directories. The campaign brief says 114. Both numbers stated; slides below use 114.

Results: from rounds to problems

Counted 2026-10-06 11:36Z from `ROUNDS.yaml` , 57 rows.



scored = SOLVED + NOT CLEARED; ABORTED rows carry no harness verdict.

chtrie scored a green workspace at the round cap (ledger SOLVED); the campaign counts it VERIFIED-WORKSPACE, a retry candidate, so the cleared count stays 13.

- One problem can span several rounds. A retry keeps its own row. A problem counts once.
- ABORTED rows are engine or launcher failures, not harness verdicts.

Ledger: `.artifacts/experiments/ROUNDS.yaml` and `SUMMARY.md` .

Coverage: 13 of 114 problems cleared



Family	Problems	Cleared
crust, C	100	11
skel (Python)	8	1
oxidizer (Go)	7	1
alphatrans (Java)	4	0

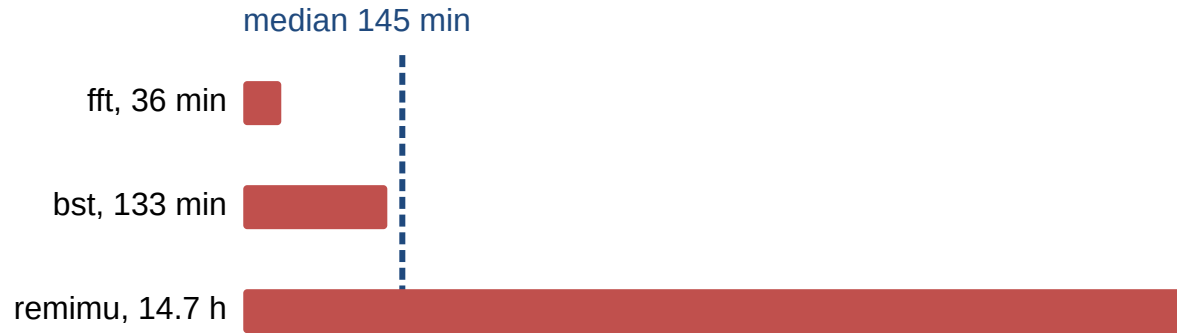
Table counts 119 problem directories on disk; the campaign brief says 114 (both stated, paper §setup); the dot grid uses the brief count.

- A clear needs the round to complete the final validation gate. chtrie stopped at the round cap with a green workspace: VERIFIED-WORKSPACE, a retry candidate, not a clear. One round in flight (strsim) at count time, 2026-10-06 11:36Z.

Ledger: `ROUNDS.yaml`, recount at build time.

Wall clock

Serial-era solved median: **145 min** (n=15). Three later failures:

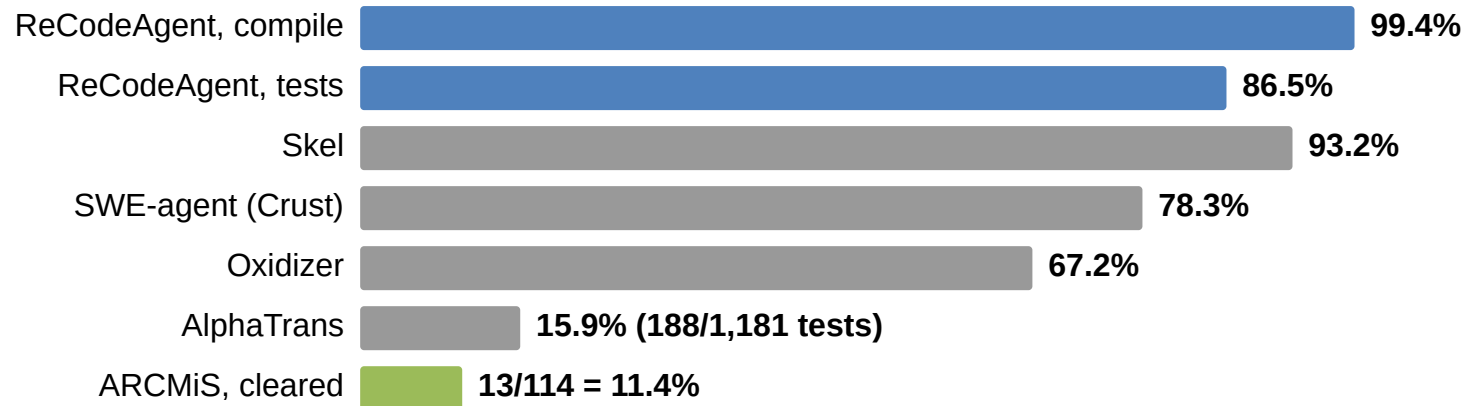


red = closed without a verified workspace · dashed line = solved median

- A failure can end fast. fft stalled at Planning and stopped in 36 minutes.
- A failure can also run long. remimu translated all five batches, hit the round cap, and still did not clear.

Ledger: `.artifacts/experiments/SUMMARY.md`, wave close notes.

Comparison with published baselines



Different protocol. Published numbers: Claude 4.5 Sonnet, 118 projects, developer-test scoring. ARCMiS: local 27B model, one set at a time, toolchain-only scoring. Directional context, not a like-for-like comparison.

Ibrahimzada, A. R., et al. (2026). ReCodeAgent. arXiv:2604.07341; baseline table verified in project report, \$baseline. AlphaTrans per Ibrahimzada, A. R., et al. (2025), FSE, as reproduced in the ReCodeAgent baseline table.

Estimated time to finish

≈ 78 days

at the current rate: 13 cleared in 10 days, 101 problems remaining.

- The rate is not constant. The sweep walks problems in ascending size order, so later problems cost more wall time per set.
- This is an estimate from the current rate, not a commitment.

Ledger: `.artifacts/experiments/ROUNDS.yaml`, recount at build time.

References

References

- Dang, D.-H., & Nguyen, T.-D. (2026). *ARCMiS: Agentic repository-level code migration for small language models*. Project report, VNU Hanoi.
- Ibrahimzada, A. R., Paulsen, B., Kroening, D., & Jabbarvand, R. (2026). ReCodeAgent: A multi-agent workflow for language-agnostic translation and validation of large-scale repositories. *arXiv preprint* arXiv:2604.07341 (ASE 2026).
- Ibrahimzada, A. R., Ke, K., Pawagi, M., Abid, M. S., Pan, R., Sinha, S., & Jabbarvand, R. (2025). AlphaTrans: A neuro-symbolic compositional approach for repository-level code translation and validation. *Proceedings of the ACM on Software Engineering (FSE)*.
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? *ICLR 2024*.
- Li, J., Miao, C., Krishnan, S., & Padman, R. (2026). JEV-as-a-judge: Accept when confident, escalate when unsure. *arXiv preprint* arXiv:2609.26550.
- Seacord, R. C., Plakosh, D., & Lewis, G. A. (2003). *Modernizing legacy systems: Software technologies, engineering processes, and business practices*. Addison-Wesley.
- Zhu, Y., Liu, L., Yu, J., & Zhang, D. (2026). LLM-based multi-agent orchestration: A survey of frameworks, communication protocols, and emerging patterns. *Future Internet*, 18(6), 326.

Project artifacts: ADRs 0018, 0022, 0023, 0026, 0027, 0028 (`docs/adr/`); dataset `assets/ReCodeAgent/data/tool_projects` ; ledger `.artifacts/experiments/ROUNDS.yaml` .

Thank you